



Transformer Embeddings

Product Roadmap

Pritha Ghosh

Daniel Xing

Matt Anderson

January 2026



Context & Motivation - Pritha

Executive Summary

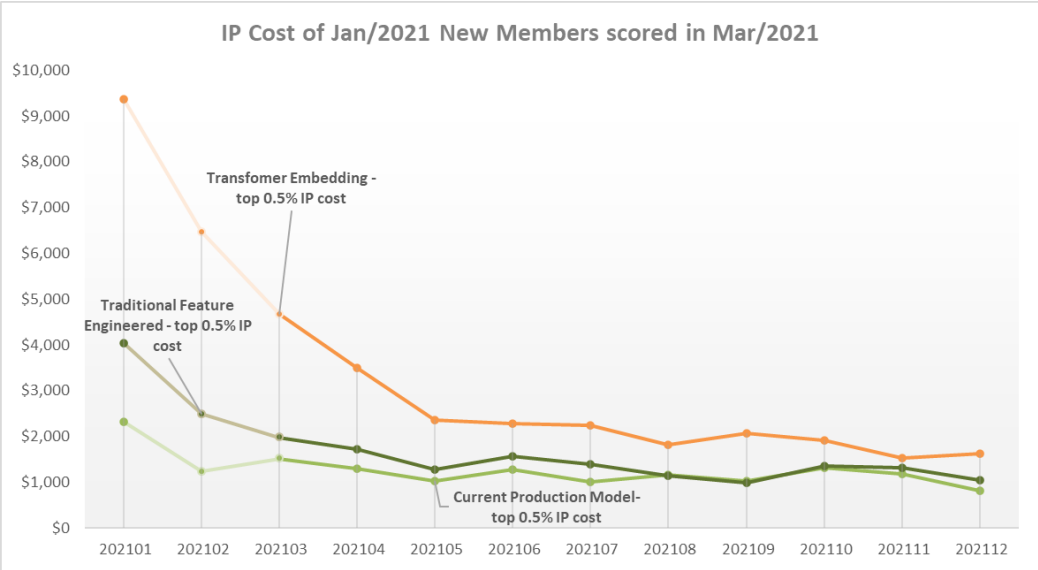
Current Reality	What We've Done	What's Next
- Transformer Embeddings (TE) have consistently shown strong performance and efficiency gains over traditional feature engineering - However, today's TE ecosystem is operationally fragmented, with separate pipelines, refresh cadences, and ownership across LOBs - The current TE foundation was trained under legacy constraints and does not support scalable, governed, cross-LOB adoption	- Established a unified "latest" TE access layer across Commercial, Medicaid, and Medicare to unblock evaluation - Created a stable consolidated dataset for downstream users while underlying pipelines remain heterogenous - Laid the groundwork for retraining on GCP, enabling evaluation, migration planning, and future standardization	Retrain TE model on a governed, cross-LOB GCP foundation - Align on inference strategy, refresh cadence, and migration approach for downstream models - Partner with downstream teams to validate performance, manage adoption, and sunset legacy embeddings

Current Evidence Snapshot

What we’ve seen so far (across prior runs) :

- TE models consistently **outperform traditional feature engineering** for high-risk member identification
- TE model identifies members **with higher IP likelihood** and **higher IP/ER** and **overall medical cost**
- Performance gains are **largest at higher-risk deciles**, where business impact is greatest
- **Standardizes feature creation from claims history**, enabling faster iteration and reducing time-to-model for new use cases

Lift at Top 1%	Commercial	Medicaid	Medicare
Traditional FE	13.6	10.2	6.1
TE only	16.4	16.9	7.3



Results from Commercial IP model

Slide curtesy of Jane Zou, Elle Palmer, Zenon Cheng

Current State: Embeddings are Fragmented Across LOBs

Commercial	Medicaid	Medicare
Nested / array-based embedding storage	Flat, column-based embedding schema	Flat, column-based embedding schema
Daily incremental + weekly full refresh	Monthly full refresh	Refreshed weekly with IP model scoring
Stored as a standalone embedding table	Standalone embedding table	Embeddings coupled with IP model features
Coupled with RAP model pipeline	Pipeline stable and refreshing as expected	Pipeline execution constrained by compute availability and design (last successful refresh: November 2025)

Key Takeaway: Embeddings exist as LOB-specific artifacts with inconsistent schemas, refresh cadences, and operational reliability – making downstream evaluation and migration difficult

What We Did: Unified “Latest” Embedding Access Layer

Unified Access Layer

- Single consolidated BigQuery view in CSDI dataset
- Brings together embeddings from Commercial, Medicaid, and Medicare
- Acts as a single access point for downstream users
- Pipelines remain owned and operated independently by LOB/model teams
- Access layer provides a stable contract across heterogeneous ownership

Design Choice

- Stores only the latest embedding per member
- Fully retrained TE product will be published to the same dataset
- Minimizes downstream change when transitioning to the unified product

Why “Latest Only” (for now)

- Serves as a bridge, not the final TE product
- Reduces operational fragility while pipelines are heterogeneous
- Avoids heavy storage and compute costs
- Enables fast evaluation on current state

Key Takeaway: This layer decouples downstream evaluation from pipeline ownership, simplifying access without changing underlying models, unblocking evaluation and migration planning

Access view here: `edp-prod-storage.edp_ent_transformer_srcv.embedding_combined`

Why Retraining Is Necessary: Rebuilding the Embedding Foundation

Retraining is not a routine model refresh – it is required to rebuild transformer embeddings on a governed, scalable GCP foundation that supports long-term enterprise use

Core Reasons for Retraining

- 1. **Establish a Governed and Reproducible Foundation**
 - Current embeddings were trained on **legacy, undocumented pipelines**
 - Training data lineage was not preserved during migration to GCP
 - Retraining allows us to:
 - Establish **traceable and auditable pipelines**
 - Ensure **reproducibility** of training and validation
 - Reduce **compliance and operational risk**
- 2. **Improve Coverage and Population Representativeness**
 - Current embeddings reflect **partial and uneven population coverage**
 - Retraining enables:
 - Inclusion of **Medicaid** for full population representation
 - Better generalization across **diverse member cohorts**
 - Alignment with **current utilization patterns**, not legacy behavior

Dimension	Current State (V2)	Retrained State (V3)
Data Coverage	Commercial, Medicare from 2016	All LOBs – Commercial, Medicare, Medicaid from 2023
Infrastructure	Legacy Hadoop, undocumented	Fully migrated and governed on GCP
Coverage	Limited data sources	Expanded to include GPI, DRG and National Provider Taxonomy
Governance	None	Full lineage, reproducibility, and metadata tracking

Key Takeaway: To move from a shared access layer to a true shared product, retraining is required to ensure governance, representativeness, and long-term scalability

Training Enablement : Infrastructure & GPU Readiness

Retraining requires upfront infrastructure and governance enablement before GPU usage is approved or feasible

What Had to Be Enabled (Before Retraining)

1. Governance & Approval Alignment

- Defined retraining scope, cost expectations, and usage boundai

Target Inference & Serving Pipeline

This pipeline defines how retrained embeddings are generated, stored, and made available for downstream use

Inference Generation	Embedding Storage	Downstream Consumption	Operational Guardrails
- Retrained embedding model is used for batch inference - Runs on a defined cadence or event trigger (claims-based) - Embeddings are generated as of a specific point in time	- Inference outputs written to CSDI dataset - Stored as time-anchored snapshots (as of index date) - Enables: - Evaluation over a fixed cohort - Reproducibility of results - Auditability	- Downstream models access embeddings via a stable CSDI interface - Consumers are decoupled from pipeline ownership	- Inference jobs are auditable and reproducible - Backfills and re-runs are supported without retraining

Key Takeaway: This pipeline supports a production TE product with phased rollout, ensuring stability and controlled adoption

Embedding Refresh Cadence – Operating Model Decision

Given the target inference pipeline, we need to align on how embeddings should be refreshed over time in a way that balances correctness, cost, and stability

What we are solving for:

- Embeddings should reflect meaningful changes in member state
- Refresh strategy should be cost-efficient and scalable
- Downstream consumers should see minimal unnecessary churn

Option 1: Fixed Time-Based Refresh

- Weekly or monthly refresh for all members

Pros:

- Simple
- Easy to schedule and monitor

Tradeoffs:

- Recomputes embeddings even when nothing has changed
- Higher ongoing compute cost

Option 2: Event-Driven Refresh (Recommended Direction)

- Update embeddings only when new claims or qualifying events arrive

Pros:

- Conceptually aligned with embeddings representing member state
- Avoids unnecessary recomputation
- Scales better as population grows

Tradeoffs:

- Requires clear definition of triggering events
- More complex orchestration

This is a product operating design that impacts cost, scalability, and downstream stability – and should be aligned across teams

Model Architecture & Training Experiments

– Daniel

Model Architecture and Refactoring

Preserve backbone

- **Hierarchical structure** for encoding within- (80 codes) and inter-day (200 days)
- Trained to predict **next-day medical codes** (multi-label, 6,297 target groups)

Training & inference efficiency

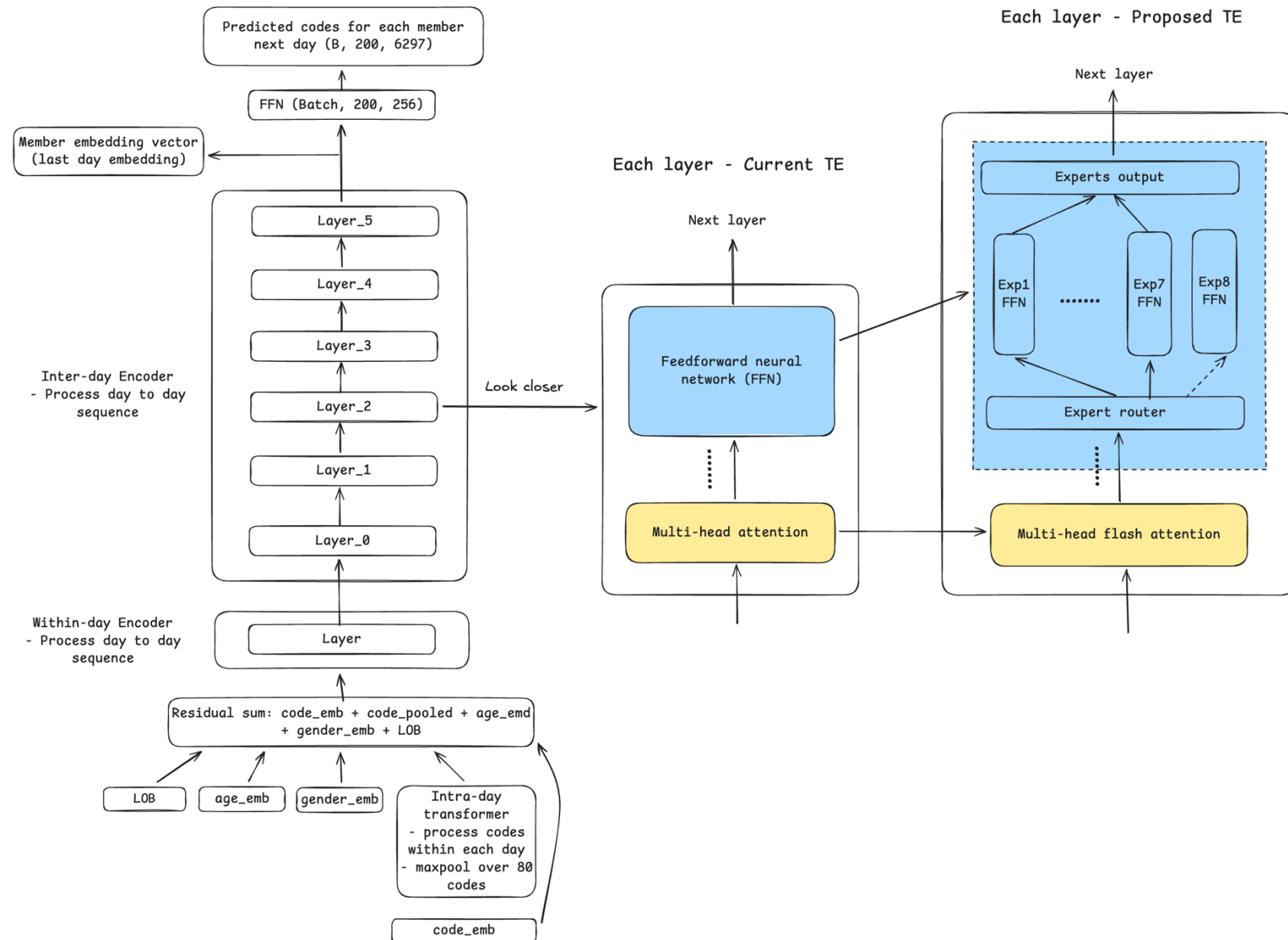
- Replace the attention layer with **flash attention** to avoid GPU IO bottleneck and provides

Specialized representation

- Replace the dense FFN with with **mixture of expert**, each token routed to 2 “experts”, improving heterogeneous feature representation without increasing cost

Optimized config

- Weighted BCE loss accounting for rare codes
- SGD -> Adam learning rate optimizer
- Head dimension 16 -> 32 (fit flash attention)



Preliminary Ablation (Intrinsic)

Data

- Commercial: 1.06M
- Medicare: 0.56M
- Medicaid: 0.13M
- Day count > 10
- Time period: 1/1/2023 – 12/31/2023
- Claim lookback period: 36 months with 90d delay

- Full input codes: 75.51k
- Target codes: 6.30k

Result

- Finding 1
- Finding 2
- Finding 3

Exp config:

- Baseline: Dense + legacy config
- Exp2: Dense + flash attention + optimized config
- Exp3: MOE + flash attention + optimized config
- GPU: T4 * 4
- Machine type

Downstream evaluation planned and lesson learnt

Downstream evaluation

- Memr

	Commercial	Medicaid	Medicare
Eval task	IP in 6 months		
Sampling strategy	(30%)		
Clinical features	Med+Rx, HPD, SDOH, Lab		
Model pipeline	Downsampling (1:10) + Selected features + Catboost		



Productionization & Next Steps - Pritha

Migration & Adoption Strategy – Open Questions for Alignment

Current Reality

- Three embedding pipelines running today
 - Owned by different Des
 - Different refresh cadences
- New embedding version implies downstream model retraining
- We cannot switch embeddings without coordination

Proposed Initial Adoption Scope

- Start with a small set of downstream models to validate
 - Commercial – IP Model
 - Medicaid – IP Model
 - Medicare – New Member Model (opportunity to test Komodo value-add)

Key Questions for Alignment:

1. **Evaluation/Validation Ownership**
 - A. TE Product Team
 - B. Individual model owners
2. **Once adoption criteria are met, how should legacy embeddings be handled?**
 - A. Define a fixed overlap window, then deprecate
 - B. Deprecate per downstream team as they migrate
 - C. Keep legacy embeddings indefinitely (not recommended)

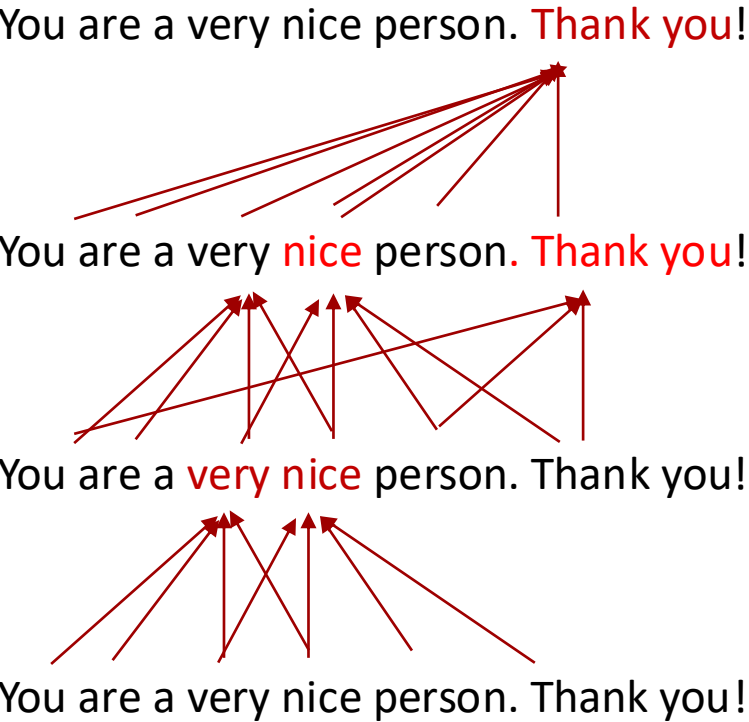


Appendix

What is transformer (cont'd)

Transformer = Stacked multi-head attention + position encoding + masking.

Stacked attention

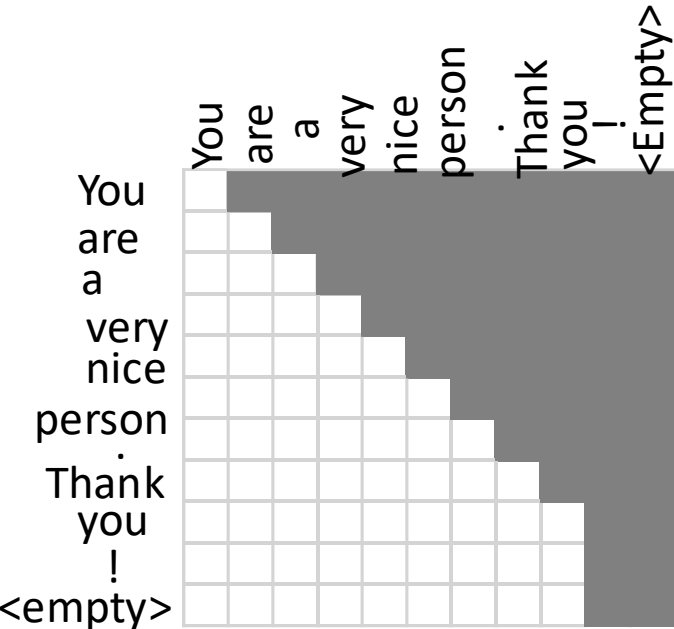


Position encoding

You	0
are	1
a	2
very	3
nice	4
person	5
.	6
Thank	7
you	8
!	9
<empty>	0

➡ Matrix 10xN

Look ahead masking (GPT)



"Attention is all you need" [Vaswani et al. \(2017\)](#)

Our transformer architecture

date	icd9_dx_cd	prcdr_cd	specialty_ctg_nm	plc_srv_ctg_cd	gpi4
2021-01-29	L30.8	99213	Pediatrics	office	6699
2021-01-29	L70.9				6846
2020-10-03	Z23	90460	Pediatrics	outpatient	
		90686			
2020-03-03	J30.9	90460	Pediatrics	outpatient	
	Z00.129	90651			
	Z13.89	90686			
	Z23	96127			
	Z68.52	96160			
	Z71.3	99173			
	Z71.82	99394			
2019-04-28	R10.9	80048	Acute Short Term Hospital	emergency room	
		81001			
		82150			
		83690			
		85025			
2019-04-27	R10.9	74176	Not Mapped	emergency room	
	V86.59XA	74176			
	Y92.838	96365			
	Y93.89	99283			
		99283			
2019-04-19					6131
					6516
2019-04-02	B34.9	87880	Pediatrics	office	
		99214			
2019-01-23	Z00.129	90460	Pediatrics	office	
	Z23	90461			
	Z68.53	90715			
	Z71.3	90734			
	Z71.89	99393			
2018-12-05					120
2018-12-03	J02.9	99203	Missing	outpatient	

Comparing to natural language, our medical data is also in sequences, but more complex:

1. Considering each date is a “word”: there may be multiple data points / types at each date, such as procedure, dx, specialty etc.
2. Dates of the records are NOT continuous.
3. Different medical datasets might not align on dates, such as pharmacy data and claim data .
4. Predicting next ”word” is more challenging than NLP.

Individual_id =49234

Our transformer architecture (cont'd)

Step1. Each date is considered as a word in a sentence. Diagnosis codes, procedure codes etc will be encoded by 1-d CNN.

date	icd9_dx_cd	prcdr_cd	specialty_ctg_nm	plc_srv_ctg_cd	gpi4
2020-03-03	J30.9	90460	Pediatrics	outpatient	
	Z00.129	90651			
	Z13.89	90686			
	Z23	96127			
	Z68.52	96160			
	Z71.3	99173			
	Z71.82	99394			

■
CNN
encoding
↓

■
CNN
encoding
↓

■
CNN
encoding
↓

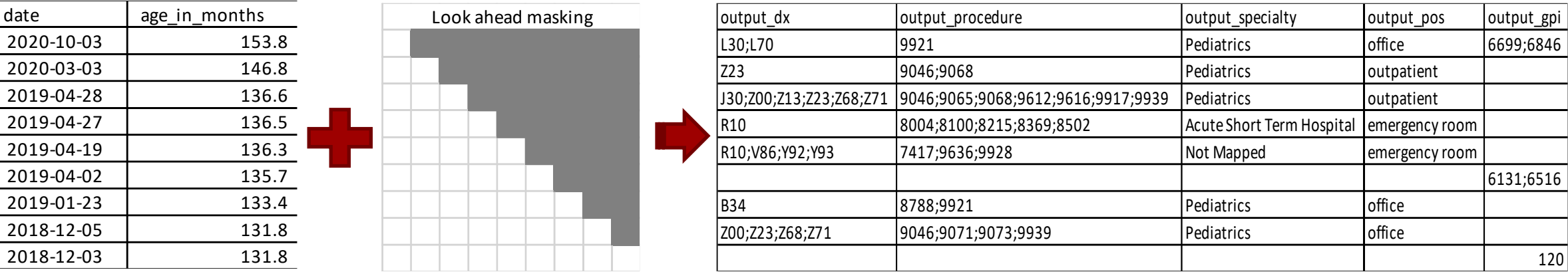
■
CNN
encoding
↓

■
CNN
encoding
↓

“word” Embedding = A + B + C + D + E

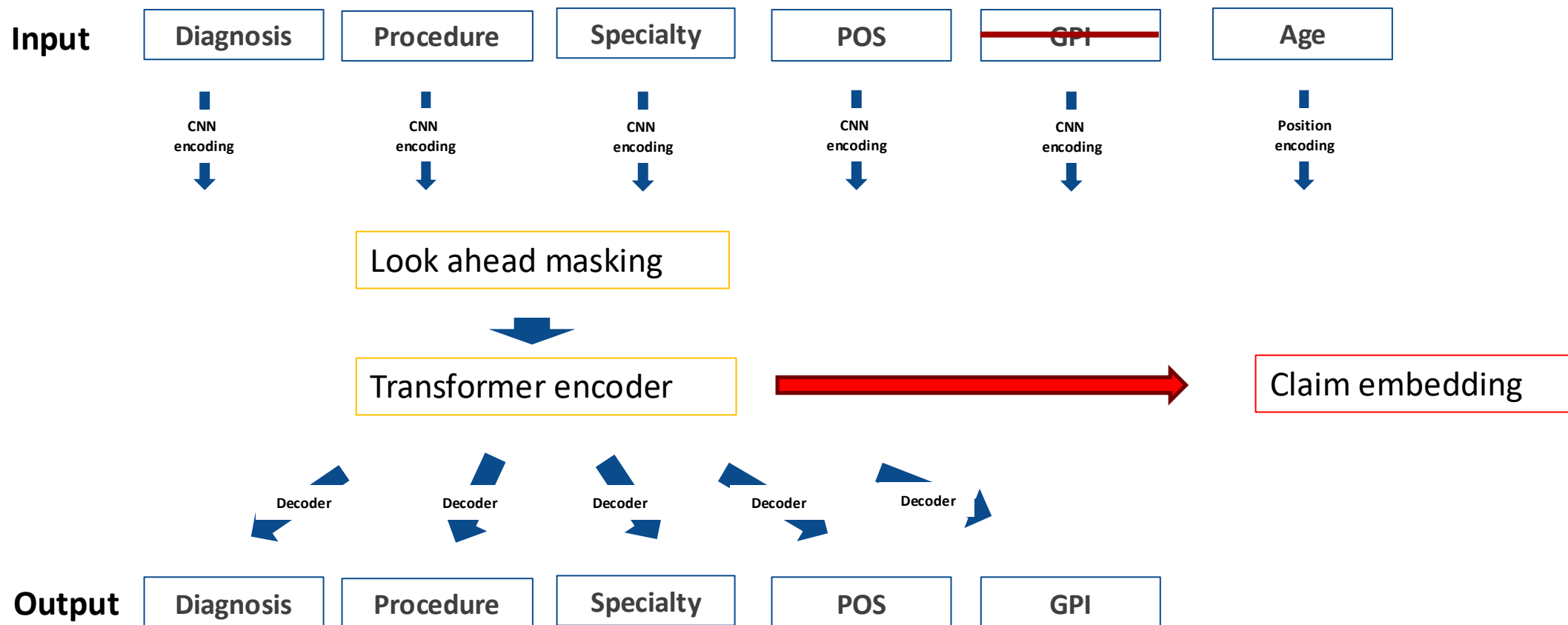
Our transformer architecture (cont'd)

Step2. Use age to encode position. Use look forward masking (like GPT), to predict next medical records, but at higher level. The padded records are ignored in loss calculation.



Our transformer architecture (cont'd)

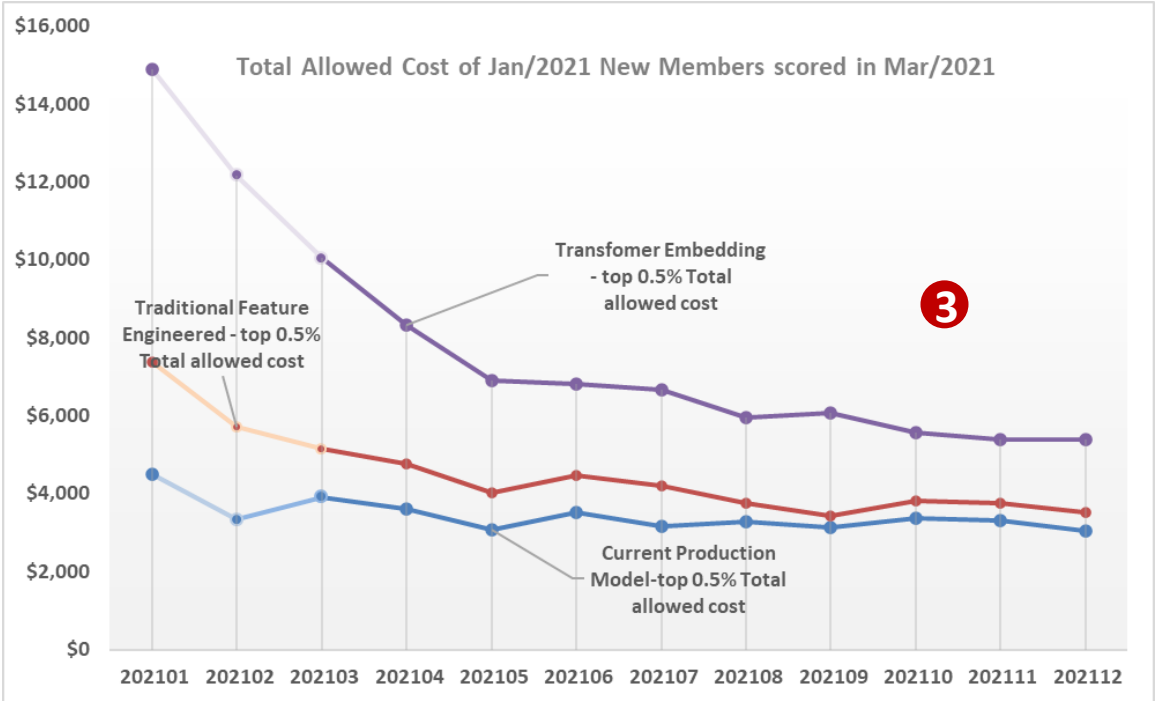
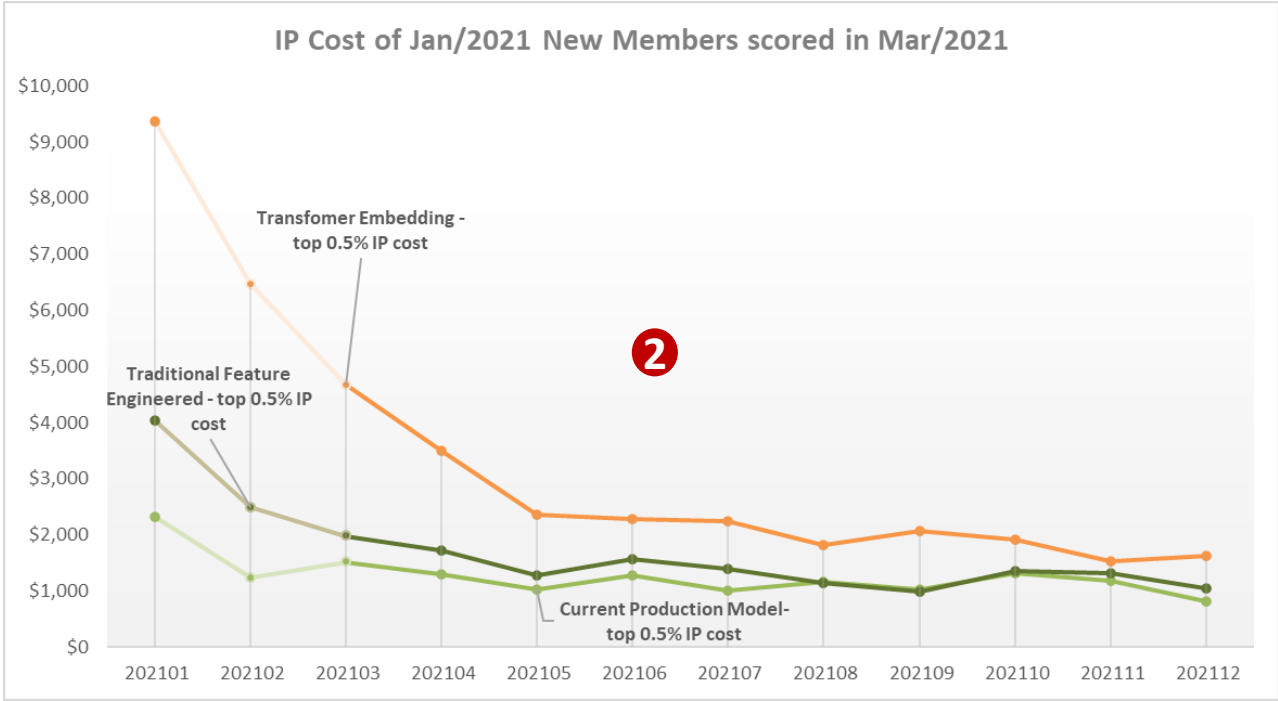
Step3. the layer right before the decoders is the embedding of the medical history, which can be used as predictors in any predictive modeling. So far we have two models: claim embedding and pharmacy embedding.



TE_(v1) model outperforms traditional feature-engineered model by wide margin

Model Lift PPV (% Improvement over Production)			
Member Selection	Benchmark Legacy	Traditional Feature Engineered	1 Transformer Embedding
Top 0.1%	21.4	27.26 (28%)	35.88 (68%)
Top 0.5%	13.5	17.67 (31%)	22.24 (64%)
Top 1%	11.1	13.67 (24%)	16.44 (49%)

- 1
- TE model and traditional feature engineered model perform better than the current model, TE model has the largest lift.
- 2
- TE model identifies members with higher IP likelihood and higher IP/ER and overall medical cost (all cost categories show same pattern)
- 3
- All 3 models indicate that member cost decreases over month after top risk identification, flattening out after month 6, suggesting urgency of care management targeting*
- *please note that due to data delay of the current process, January enrolled new members are scored in March, EDW data up to Feb is used for the March scoring, so we should look at trend starts from March



Medicare IP model with TE (v2)

Comparisons - based on OOT Validation		1%			10%		
	AUC	PPV	Sensitivity	Lift	PPV	Sensitivity	Lift
XGB - NRX V2 (default params)	0.78	49.0%	7.4%	8.16	25.2%	38.3%	4.20
XGB - All Features V2	0.775	48.9%	7.7%	8.15	24.4%	38.6%	4.06
XGB - All Features (default params)	0.775	48.6%	7.7%	8.10	24.3%	38.4%	4.05
Catboost - All Features	0.773	48.5%	7.7%	8.09	24.1%	38.2%	4.02
XGB - RX V2 (default params)	0.771	49.2%	7.9%	8.20	24.0%	38.5%	4.00
XGB - Embeddings + CS (minus Med/RX Claims)	0.771	47.4%	7.5%	7.90	23.9%	37.8%	3.99
XGB - NRX V2 CS Only (default params)	0.771	48.3%	7.3%	8.05	24.3%	36.8%	4.05
XGB - CS Features V2	0.768	47.3%	7.5%	7.88	23.5%	37.3%	3.92
Catboost - Embeddings + CS (minus Med/RX Claims)	0.768	47.1%	7.5%	7.86	23.8%	37.7%	3.97
XGB - CS Only (default params)	0.766	47.0%	7.4%	7.83	23.4%	37.0%	3.90
Catboost - CS Only	0.765	46.7%	7.4%	7.79	23.4%	37.0%	3.90
XGB - RX V2 CS Only (default params)	0.764	46.9%	7.5%	7.82	23.0%	34.8%	3.83
XGB - Embeddings (v2) Only	0.748	45.1%	7.1%	7.39	23.4%	36.6%	3.82
XGB - HPD Only (default params)	0.738	-	-	-	-	-	-
Production	0.732	38.1%	6.1%	6.10	20.9%	33.2%	3.34
XGB - Embeddings (v1) Only	0.717	35.8%	5.6%	6.20	20.4%	31.9%	3.36

- Embeddings did not perform as well as traditional feature engineering here, due to one key problem- RX (GPI4) was absent from the embeddings, which is crucial for the Medicare population.
- We would expect a boost to performance for embedding-only models, and perhaps a small boost to combined feature models.

Medicaid IP model with TE_(v2) preliminary findings

- The TE model was trained in commercial and Medicare but still performs well in Medicaid
- Current best model in development has an AUC of ~0.87 and lift of ~20 in top 1% (data not shown)

25% holdout embeddings vs. Product Spec (Medicaid)		1%			10%		
	AUC	Precision	Recall	Lift	Precision	Recall	Lift
Logistic Regression - Embeddings only (6 month)	.84	28.9%	17%	16.98	10%	58.1%	5.81
Production (12 month)		45.1%	10.3%	10.29	19.6%	44.6%	4.46

- Breakout by tenure suggests a new member model in Medicaid is not necessary

Logistic Regression – Embeddings only (by membership length)		1%			10%		
	N (%)	Precision	Recall	Lift	Precision	Recall	Lift
0-3 Months	35,354 (6.2%)	19.8%	10.8%	10.79	6.8%	36.9%	3.69
4-8 Months	55,853 (9.8%)	26.7%	17.8%	17.77	8%	53.5%	5.35
9+ Months	477,535 (84%)	29.9%	17.8%	17.83	10.3%	61.6%	6.16

TE empowers data scientists to build better models more efficiently

Better performing model

- **Better lift/precision/AUC**
- **TE captures the intricacy of our data**, both structured and unstructured in a sequential manner, generating a condensed representation that encompasses causal effects and enhances our predictive models. This will be harder to achieve manually with thousands of sparse features. Traditional feature selection often drops features with low prevalence, such as pancreatic cancer, TE can capture rare and costly diseases.
- **Bigger business impact** Early experiments showed TE model identifying higher risk members (more IP/ER utilization, higher spend)

More Efficient

- **Time-saving**. As a data scientist, we spend up to 80% of our time cleaning and transforming data to generate actionable insights and build machine learning models.
- **Standardizes and streamlines process with claim history data**. It simplifies data scientists' work, saves **weeks**' time and system resources, allowing us to focus on applying the latest ML skills and building models that effectively address our business goals, solve problems, and deliver value to our customers.